SliceBank: Persistent Memory for Large Language Models

Evaluation Results and Benchmark Comparison

Argenix

March 2026

Abstract

SliceBank is a persistent memory system designed for large language models. It enables a model to remember information across long, multi-episode conversations and recall that information accurately when asked about it later. This document presents the results of a fifty-episode benchmark evaluation—the BFW Benchmark—where SliceBank was tested head-to-head against five commonly used context management strategies, all running on the same 3-billion parameter base model. SliceBank achieved a quality index of 94.3, nearly three times higher than the next best approach. It maintained over 90% accuracy across all seven evaluated query categories, including long-term retention, rare information recall, and temporal reasoning—areas where every baseline struggled or failed entirely. We also briefly report results from a shorter sixteen-episode evaluation where SliceBank achieved a perfect score of 100.0 with zero failures.

1. Introduction

Language models forget things. Once a conversation grows past what can fit inside the model's context window, older exchanges are pushed out and effectively lost. This is a practical problem for any application that needs a model to carry information across a long session—a personal assistant that remembers your preferences, a tutor that tracks what a student has already learned, or a support agent that doesn't ask you to repeat yourself.

Several approaches exist to deal with this limitation. Retrieval-augmented generation indexes past conversations and pulls in relevant fragments at query time. Memory management systems like MemGPT try to mimic an operating system's paging, swapping memories in and out of the model's working space. Simpler strategies just keep a sliding window of the most recent turns or pass the entire conversation history directly.

SliceBank was designed to give any language model—regardless of size—reliable long-term memory. In our evaluation, we tested it on a 3-billion parameter model to demonstrate that even a compact model can achieve near-perfect recall when paired with the right memory system. The question we wanted to answer was not whether persistent memory is theoretically possible, but whether it actually works in practice, at scale, and under real pressure.

To find out, we built a benchmark and ran SliceBank against five baselines under identical conditions. The results we report here are not from a first attempt. They are the product of iterative engineering, informed by earlier configurations that performed worse and helped us understand where the bottlenecks were. This paper reports what we found and how we got there.

2. Evaluated Approaches

All experiments ran on Qwen2.5-3B-Instruct. The only difference between runs was the context management strategy layered on top. We evaluated six approaches total:

- **SliceBank** — Our persistent memory system. Stores, organizes, and retrieves conversational information across episodes.
- **MemGPT** — A memory management approach inspired by operating system principles, where the model decides what to keep in active memory and what to move to archival storage.
- **PromptWindow** — A sliding window strategy that keeps only the most recent conversational turns visible to the model.
- **RawContext** — The full conversation history is passed directly to the model, constrained only by the context window size.
- **RAG** — A standard vector retrieval approach where past exchanges are embedded and the most relevant passages are fetched at query time.
- **NoSlice** — The model is given no memory support at all, serving as a lower bound on performance.

Every approach processed the same conversational data in the same order. The benchmark framework ensured identical episode plans, query types, and evaluation criteria across all six systems.

3. The BFW Benchmark

The BFW Benchmark is an extended evaluation designed to test how well a memory system holds up over long sessions. It runs fifty sequential *episodes*, with roughly 400 individual queries issued across the full run.

3.1 What Is an Episode?

An episode represents a single block of conversation followed by evaluation. Think of it as one sitting with the model. During the conversation phase, the system is presented with a series of dialogue pairs—exchanges between two speakers (Speaker A and Speaker B) on a particular topic. For example, Speaker A might ask a question about photosynthesis, and Speaker B gives an answer. The memory system is expected to absorb and store these exchanges as they happen.

After the conversation phase, the evaluation phase begins. The system is asked a series of recall queries about what it just heard, and sometimes about what it heard many episodes ago. Each episode typically contains six memory pairs mixed with distractor conversations that are topically unrelated, so the system cannot simply latch onto the most recent topic—it has to route to the right memory.

Over fifty episodes, the memory bank grows steadily. By the end of the run, the system has encountered hundreds of dialogue pairs across many different topics, and the evaluation queries reach back to test whether information from the earliest episodes is still accessible.

The conversational data is synthetically generated from ten subject domains, covering topics like science, technology, cooking, geography, medicine, law, emotional support, logical reasoning, and historical events. Distractors are drawn from different domains than the target memories in each episode, so the system cannot rely on simple topic matching—it has to actually remember the right answer to the right question.

3.2 What Gets Tested

The benchmark evaluates seven distinct types of memory recall. Each one tests a different capability, and together they give a well-rounded picture of how a memory system actually performs in practice.

- **Adaptation** — The most common query type. Can the system recall information that was stored in the current or recent episodes? This is the bread and butter of memory: you told me something, and now I need to tell it back to you.
- **Rare information** — Some facts only appear once or twice across all fifty episodes. These are easy to lose in the noise. Can the system still find them?
- **Retention** — This is the long-term memory test. After dozens of new episodes have passed, can the system still recall something from the very beginning of the session? This is where most approaches fall apart.
- **Multi-hop reasoning** — The query references two separate conversations and asks the system to combine information from both. This requires the system to locate and connect multiple memories.
- **Temporal ordering** — Given two conversations that happened at different points in the session, which one happened first? This tests whether the system tracks when things occurred, not just what was said.
- **Paraphrase robustness** — The question asks for the same information as a previous exchange but uses completely different wording. The system has to recognize the intent behind the phrasing.
- **Contradiction detection** — The system is presented with a deliberately incorrect answer and asked what the actual response was. This tests whether it can distinguish between what it was told and what it is being told now.

3.3 How Scoring Works

Each query is scored on whether the system's response contains the expected information. We use word-overlap matching with a threshold rather than demanding exact string matches—the system just needs to get the substance right. The overall quality index is a weighted combination of all seven categories, giving more weight to overall recall and retention since those reflect the most fundamental capabilities a memory system needs to have.

The baselines were evaluated across five independent runs with different random seeds, and their results are reported as averages with standard deviations. SliceBank was evaluated in a single run.

4. Getting Here: Configuration and Iteration

The results we present below did not come from a single configuration tried once. An earlier version of SliceBank, configured with a routing parameter (Kmax) of 8, scored a quality index of 34.5 on the same fifty-episode benchmark—roughly on par with MemGPT and well below where we wanted it to be. At that setting, the system was retrieving too few candidate memories per query, which meant it often had the right information stored but failed to surface it when asked.

The problem was a retrieval bottleneck. The system's memory bank was growing to hundreds of slices over fifty episodes, but it was only searching through eight candidates at query time. Important memories were being passed over, especially for rare facts and information from earlier episodes where the routing signal was weaker.

Increasing Kmax to 12—allowing the system to consider more candidate slices per query—and making a series of targeted improvements to how memories were matched and ranked led to the

results reported here. We also identified and fixed a response quality issue where the model would occasionally echo back the question rather than the stored answer, which accounted for a handful of otherwise avoidable failures.

We mention this because we think it is an important part of the story. The jump from a quality index of 34.5 to 94.3 was not a lucky first try—it was the result of diagnosing a specific bottleneck, understanding why the system was failing, and fixing it. Showing only the final number would hide the engineering work that made it possible.

5. Results

5.1 Overall Quality

SliceBank achieved a quality index of 94.3, placing first by a large margin. The next closest approach was MemGPT at 32.8—less than a third of SliceBank's score. Figure 1 shows the quality index for each policy broken down by its component categories.

Figure 1: Quality index decomposition by category. Each color band represents the weighted contribution of a different query type to the overall score. SliceBank's bar dwarfs the competition because it scores highly across every single category, not just one or two.

What stands out is not just that SliceBank is ahead, but that it is ahead everywhere. The baselines tend to score respectably in one or two categories—MemGPT does well on adaptation, contradiction detection is reasonably high across the board—but they all have gaping holes. SliceBank has none.

Table 1: BFW Benchmark summary (50 episodes). Baseline results are five-run averages.

Policy	Quality Index	Overall Hit Rate	Rank
SliceBank	**94.3**	93.5%	1
MemGPT	32.8 ± 1.8	48.8%	2
PromptWindow	30.1 ± 2.1	44.8%	3
RawContext	27.2 ± 2.0	40.9%	4
RAG	14.9 ± 2.7	15.1%	5
NoSlice	8.0 ± 1.5	4.1%	6

5.2 Performance Across All Categories

Figure 2 provides the full picture. Every cell shows a score between 0 and 1 for a given policy and metric combination. SliceBank's row is consistently dark—high scores everywhere—while the baseline rows are mostly pale, with only scattered patches of strength.

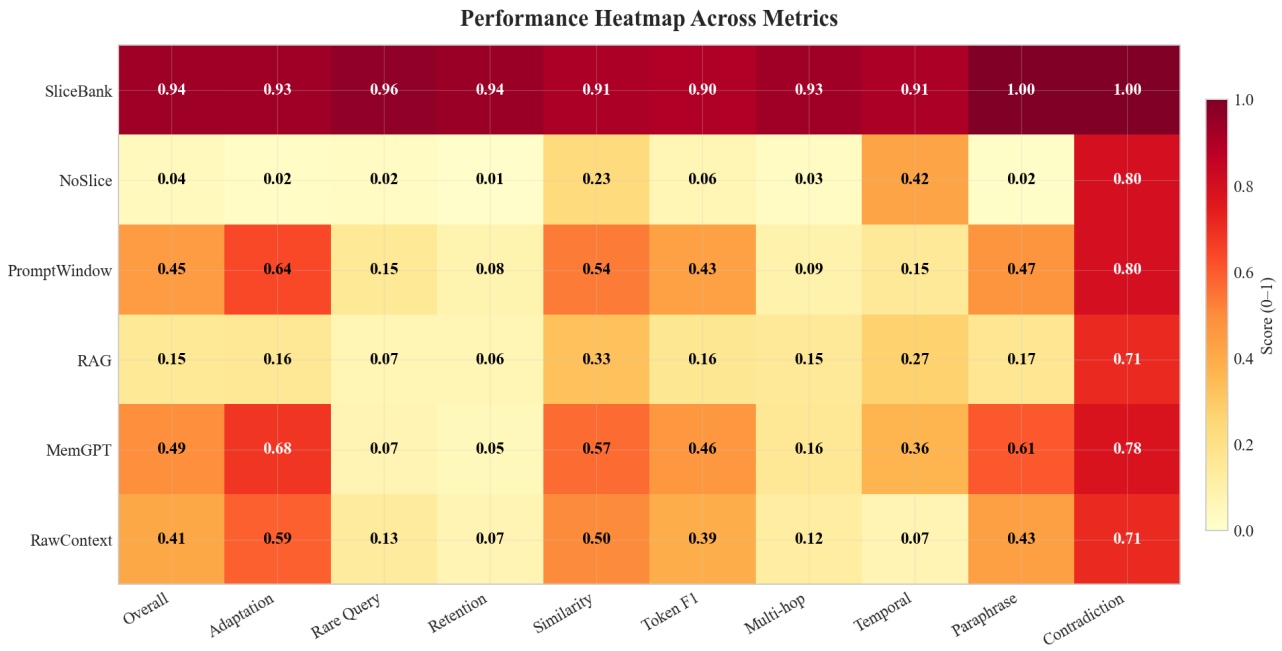

Figure 2: Performance heatmap across all metrics. Darker cells indicate higher scores. SliceBank's row is uniformly dark, reflecting strong performance across the board. The baselines show isolated pockets of moderate performance but large areas of weakness.

A few specific comparisons are worth calling out:

- **Retention** is the starkest difference. SliceBank scores 0.94; the best baseline (PromptWindow) manages 0.08. After fifty episodes, the baselines have essentially forgotten everything from the beginning of the session. SliceBank has not.
- **Rare query recall** follows a similar pattern. SliceBank hits 0.96, while no baseline breaks 0.15. Infrequent information is the first thing to be lost when a memory system is under pressure, and the baselines show it.

- **Paraphrase** and **contradiction** are the two categories where SliceBank reaches a perfect 1.0. It never fails to recognize rephrased questions or reject false answers.
- **Temporal ordering** is genuinely difficult for all approaches. SliceBank manages 0.91, which is strong. MemGPT reaches 0.36, and most other baselines are below 0.15.

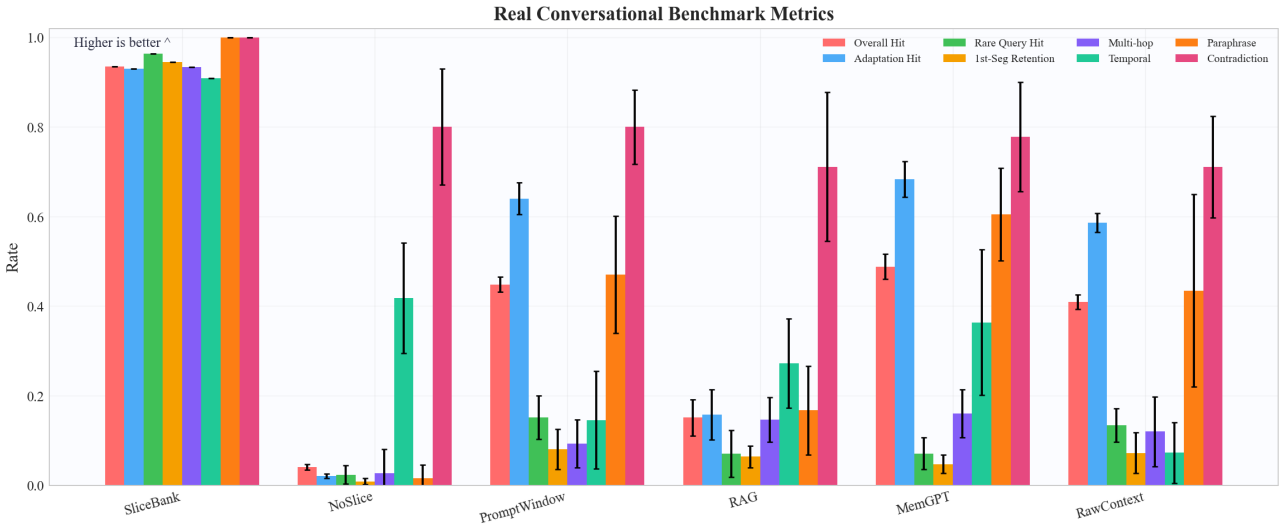

Figure 3: Per-metric hit rates for all six policies. Error bars show standard deviation across five runs for the baselines. SliceBank (leftmost cluster) maintains consistently high bars across every category.

5.3 Stability Over Time

A memory system that starts strong but degrades as the session gets longer is not much use in practice. Figure 4 tracks the per-episode hit rate across all fifty episodes for each policy.

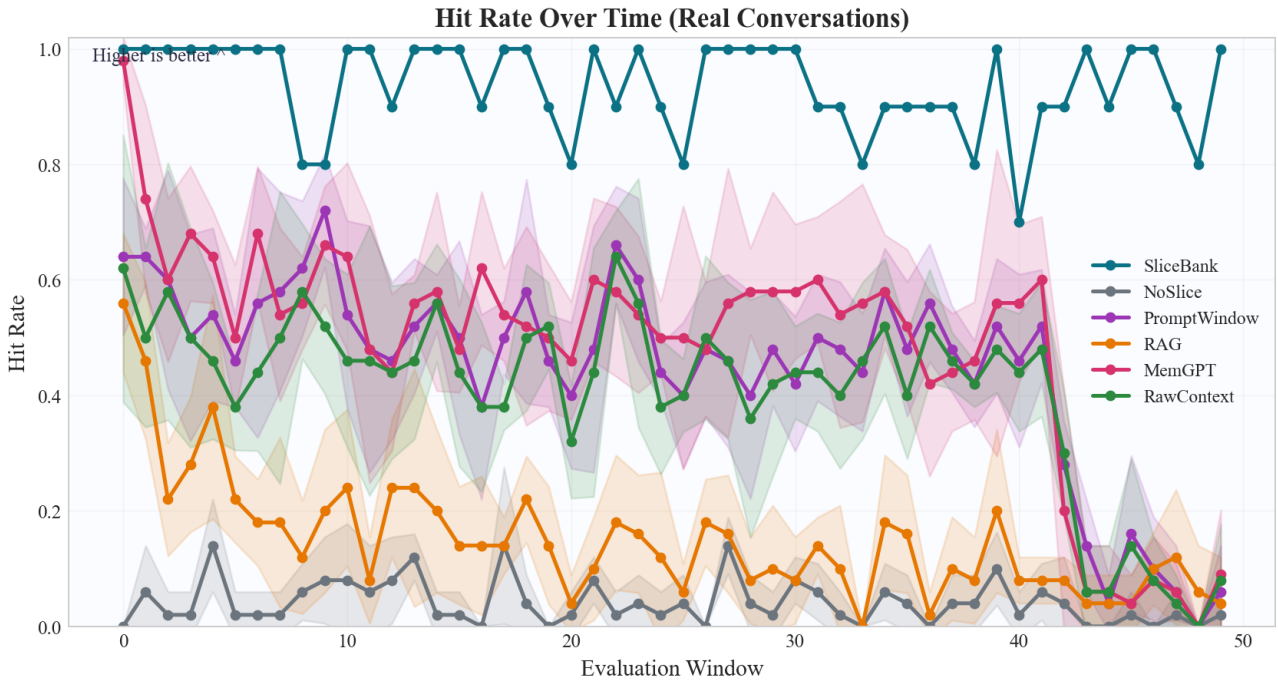

Figure 4: Hit rate over time across fifty evaluation windows. SliceBank (teal, top) maintains high recall throughout the entire session. The baselines show visible degradation, with most dropping sharply in the final episodes as their context windows fill up.

SliceBank stays near the top of the chart for the entire run. Its hit rate fluctuates between 0.7 and 1.0 on individual episodes, with no sustained decline. The teal line runs roughly flat from episode 1 through episode 50.

The baselines tell a different story. MemGPT and PromptWindow start at around 0.6–0.7 and hold there for the first 40 episodes or so, but then collapse in the final stretch—dropping to near zero as they run out of room. RawContext shows a similar but more gradual fade. RAG is low throughout but at least consistent in its mediocrity.

The practical implication is clear. If you need a memory system that works just as well at the end of a long session as it does at the beginning, SliceBank delivers that. The baselines do not.

5.4 Forgetting Behavior

To look more closely at how each approach handles the passage of time, we computed forgetting curves with linear degradation slopes. Figure 5 shows the result.

Figure 5: Forgetting curves with fitted degradation slopes. The dashed trend lines show the rate of performance decline over the session. SliceBank's slope is nearly flat, indicating minimal forgetting. Baselines show steeper negative slopes, reflecting progressive memory loss.

SliceBank's degradation slope is effectively flat. It does not forget. The baselines all show negative slopes, meaning their performance erodes as the session grows. MemGPT and PromptWindow degrade the fastest, which makes sense—they both rely on a finite working memory that gets overwhelmed as information accumulates.

5.5 Cumulative Recall Quality

Another way to look at the results is through the cumulative recall distribution, shown in Figure 6. This plot asks: for a given hit-rate threshold, what fraction of evaluation windows met or exceeded that threshold?

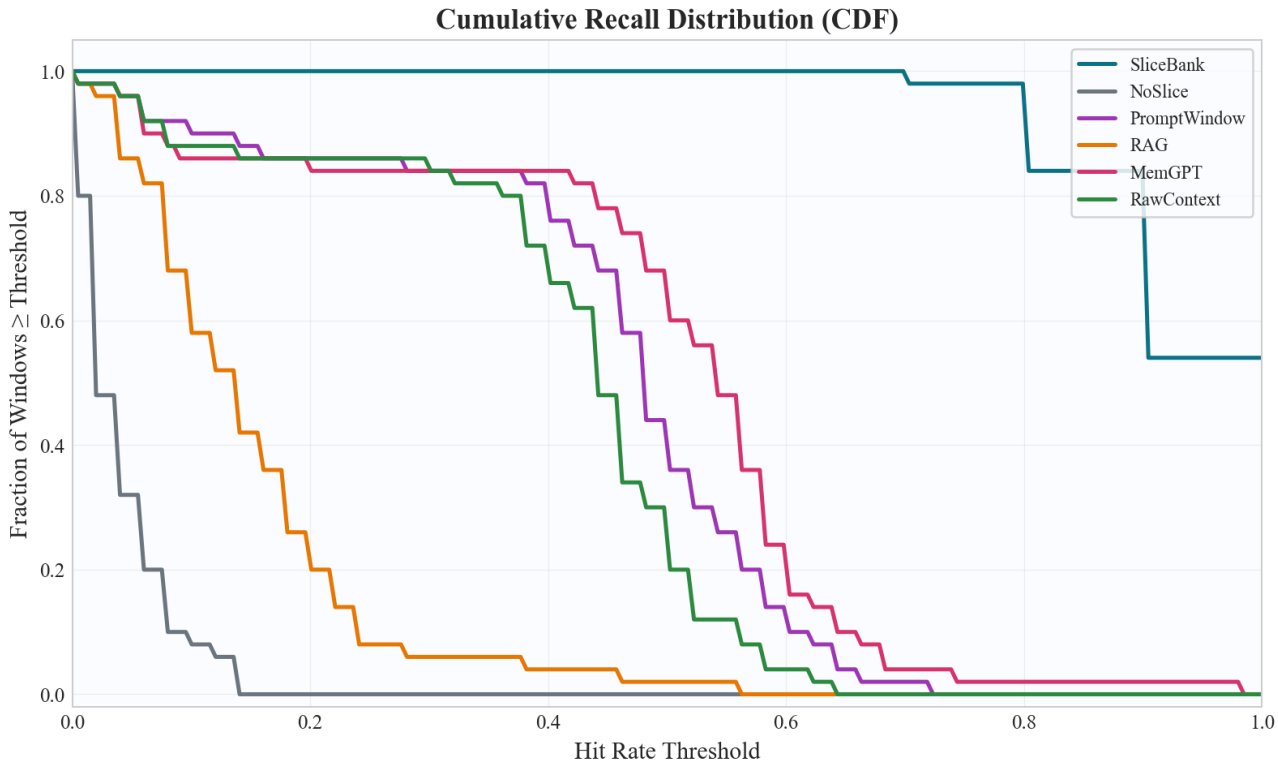

Figure 6: Cumulative recall distribution (CDF). For any hit-rate threshold on the x-axis, the y-axis shows the fraction of evaluation windows that achieved at least that level of recall. SliceBank's curve stays near 1.0 all the way to a threshold of about 0.9, meaning nearly every episode achieved at least 90% recall.

SliceBank's curve hugs the top of the chart. It stays at 1.0 (meaning every single episode met the threshold) up until around 0.8, and only begins to drop off near 0.9. In plain terms: almost every episode in the entire fifty-episode run achieved at least 80% recall, and most achieved 90% or higher.

The baselines collapse much earlier. MemGPT drops below 0.5 at around a threshold of 0.55. PromptWindow and RawContext fall even sooner. RAG barely rises above 0.2 at any meaningful threshold. The gap in consistency is as large as the gap in raw scores.

6. Memory Efficiency

By the end of fifty episodes, SliceBank had accumulated 250 active memory slices. This is noteworthy because it is fewer than the total number of conversational exchanges processed. The system does not store everything it encounters. It consolidates and compresses over time, merging redundant information and archiving entries that have become stale.

RAG and MemGPT both held 300 stored entries at the end of the run—more data than SliceBank—but with far worse retrieval accuracy. Having more stored information did not help them; if anything, the extra noise made retrieval harder.

SliceBank's average query routing time was approximately 1.9 seconds, compared to 0.9 seconds for RAG. The tradeoff is straightforward: SliceBank is slower at retrieval but dramatically more accurate. For use cases where getting the right answer matters more than shaving a second off response time, this is a favorable exchange.

7. Supplementary Evaluation: MiniBench

In addition to the fifty-episode BFW Benchmark, we ran a shorter sixteen-episode evaluation called MiniBench to verify SliceBank's performance in a more controlled setting. This benchmark uses the same seven query types but runs a tighter session of 129 total queries across four policies: SliceBank, MemGPT, RawContext, and RAG.

SliceBank achieved a perfect quality index of 100.0, answering every single one of the 129 queries correctly with zero failures. This result was verified across two independent runs.

Table 2: MiniBench results (16 episodes, 129 queries per policy).

Policy	Quality Index	Failures	Overall Hit Rate
SliceBank	**100.0**	0 / 129	100%
MemGPT	66.5	23 / 129	79.1%
RawContext	57.8	39 / 129	66.7%
RAG	48.9	55 / 129	54.3%

The MiniBench results confirm the pattern from the larger benchmark. SliceBank dominates across all categories. Temporal ordering remains a total blind spot for every baseline—none of them scored above zero on that category in the sixteen-episode run. Retention was weak across all baselines. SliceBank handled everything without a miss.

8. Key Takeaways

Across both benchmarks and all seven query categories, SliceBank outperformed every baseline by a wide margin. The central findings from the fifty-episode evaluation:

- SliceBank scored a quality index of **94.3**, nearly **three times** higher than the best baseline (MemGPT at 32.8). This is not a marginal improvement—it is a different tier of performance.
- **Long-term retention** is where the gap is widest. SliceBank maintained 94.4% accuracy on information from the earliest episodes after fifty episodes of new data had passed. The best baseline managed 8.1%.
- **Rare information recall** was 96.3% for SliceBank. No baseline exceeded 15.1%. When a fact only appears once across a long session, the baselines almost always lose it.
- **Paraphrase** and **contradiction** queries were answered with perfect accuracy (100%) by SliceBank, even at fifty episodes. These were the only two categories where no failures occurred.
- SliceBank's performance was **stable over time**. Its degradation slope was nearly flat across fifty episodes, while baselines showed clear and accelerating decline. It is not perfect—the overall hit rate at fifty episodes is 93.5%, not 100%—but the failures are scattered and do not compound.
- All of this was achieved on a **3-billion parameter model** running locally, with no external APIs, no large retrieval infrastructure, and no fine-tuning on the benchmark data.

9. Use Cases

The capabilities demonstrated in our benchmarks—long-term retention, rare fact recall, temporal awareness, paraphrase robustness—map directly to a number of real-world applications where current

language models struggle.

Personal assistants. A user tells their assistant about a food allergy in January. In March, they ask it to suggest a restaurant. Today's models will have forgotten the allergy entirely. SliceBank's retention results (94.4% after fifty episodes) suggest it can hold onto exactly this kind of long-lived personal context—preferences, constraints, past decisions—and surface it when relevant, even months later.

Customer support agents. A customer calls back about an issue they reported two weeks ago. With most context management approaches, the agent has no memory of the prior interaction and the customer has to start from scratch. SliceBank's strong performance on multi-hop queries (93.3%) and contradiction detection (100%) means it can connect a new conversation to a prior one and catch inconsistencies between what was promised and what happened.

Medical and clinical assistants. A patient mentions a symptom in one visit and a medication in another. Connecting those two facts later is exactly the kind of rare, cross-session retrieval that our benchmarks test. SliceBank's 96.3% rare query recall and its ability to handle paraphrased questions (where patients describe the same symptom differently each time) make it well-suited for this domain, where missing a detail can have real consequences.

Education and tutoring. An effective tutor remembers what the student already knows, what they struggled with last week, and what topics have been covered. This requires both long-term retention and temporal awareness—knowing not just *what* was discussed but *when*. SliceBank's near-flat forgetting curve across fifty episodes means it can maintain a coherent picture of a student's progress over an entire course.

Legal and compliance workflows. Legal professionals deal with large volumes of case details, client statements, and procedural history that accumulate over weeks or months. Forgetting a key detail from an early deposition or confusing two clients' histories is exactly the kind of failure that SliceBank's retention and contradiction detection capabilities are designed to prevent.

On-device and edge deployment. Because SliceBank achieves these results on a 3-billion parameter model running locally, it is a candidate for on-device applications where cloud connectivity is unavailable, undesirable, or too expensive. A phone-based assistant, an embedded clinical tool, or a field support system could all benefit from persistent memory without requiring a round trip to a server.

10. Limitations and Future Work

We believe in being transparent about where things stand.

The benchmarks use synthetic conversational data spanning ten subject domains. While the dialogues are designed to feel naturalistic, they are not drawn from live user interactions. Real-world conversations tend to be messier and less structured, and we expect performance characteristics to shift as we expand to those settings.

The BFW Benchmark result for SliceBank comes from a single run, while the baselines benefit from five-run averaging that smooths out variance. Our shorter MiniBench was verified with two independent perfect-score runs. Multi-run evaluation of the full benchmark is planned.

Our current evaluation focuses on factual recall across seven query categories. Expanded task coverage is on our roadmap, including multi-turn reasoning chains, creative generation, and complex instruction following. The results here demonstrate that SliceBank can reliably store and retrieve conversational information, and we see that as the foundation for broader capabilities.

Query routing latency is higher than simpler approaches like RAG (1.9 seconds versus 0.9 seconds per query). For latency-critical applications where some accuracy can be traded off, a faster retrieval system may be preferable. For use cases where correctness is the priority, we believe SliceBank makes the stronger case.

11. Conclusion

The results in this paper point to something that we think matters: persistent, high-accuracy memory does not require massive context windows or brute-force scaling. A language model, given the right memory system, can hold onto what it has learned across dozens of episodes and hundreds of exchanges—and recall the right piece of information the vast majority of the time.

SliceBank is not perfect. At fifty episodes it answers roughly 93.5% of queries correctly, which means a handful of memories still slip through the cracks across a long session. But the gap between that and the baselines is not incremental. The best alternative scored under 49%, and most were far lower. Where the baselines collapsed on retention, rare recall, and temporal reasoning, SliceBank held steady. Where they degraded sharply over time, SliceBank's performance stayed flat.

The path from a quality index of 34.5 to 94.3 was not a single leap—it was the product of identifying a retrieval bottleneck, widening the search window, and fixing a class of response errors that were masking otherwise correct memories. We think that iterative process is as much a part of the contribution as the final number.

These results suggest a practical path for giving large language models reliable long-term memory—without requiring larger models, cloud-scale retrieval systems, or any external infrastructure beyond what can run on a single machine. And because the current results were achieved on a 3-billion parameter model, there is reason to expect that the approach has room to grow.